

1. Importing Libraries

```
import json
import numpy as np
import pickle
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Embedding, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import classification_report, accuracy_score
```

- **json** → To read your dataset from a JSON file.
- **numpy (np)** → For numerical operations like arrays.
- **pickle** → To save Python objects like the tokenizer and classes.
- **TensorFlow/Keras** → For building and training the neural network (LSTM).
- **sklearn.metrics** → To evaluate the model's accuracy and performance.

2. Load Dataset

```
with open("clothes_data.json", "r", encoding="utf-8") as file:
    data = json.load(file)
```

- Reads your chatbot dataset (patterns & responses) from a JSON file.
- data now contains all intents, tags, patterns, etc.

3. Prepare Data

```
sentences = []
labels = []
```

```

classes = []

for intent in data["intents"]:
    tag = intent.get("tag")
    patterns = intent.get("patterns", [])

    if not tag:
        print("✗ Missing tag in intent:", intent)
        continue

    for pattern in patterns:
        sentences.append(pattern.lower()) # Convert to lowercase
        labels.append(tag)

    if tag not in classes:
        classes.append(tag)

classes = sorted(classes)

```

- Extracts all **patterns (user inputs)** and their **tags (labels)**.
- Stores unique tags in `classes`.
- Example:
 - Pattern: "What sizes are available?" → tag: "size_query"

4. Tokenization

```

tokenizer = Tokenizer(oov_token="<OOV>")
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index

```

- Converts words into numbers (tokenization) so the model can understand them.
- `<OOV>` token is used for words not in the training data.

```

sequences = tokenizer.texts_to_sequences(sentences)
max_len = max(len(seq) for seq in sequences)

```

```
X = pad_sequences(sequences, maxlen=max_len, padding="post")
```

- **texts_to_sequences** → Each sentence becomes a list of numbers.
- **pad_sequences** → Makes all sentences the same length (necessary for LSTM).

5. Encode Labels

```
label_index = {c: i for i, c in enumerate(classes)}  
y = np.array([label_index[label] for label in labels])  
y = to_categorical(y, num_classes=len(classes))
```

- Converts string labels (tags) into **numbers**.
- `to_categorical` → Converts numbers into **one-hot vectors** for multi-class classification.

Example:

- "size_query" → [0, 1, 0, 0, ...]

6. Save Tokenizer & Classes

```
pickle.dump(tokenizer, open("tokenizer.pkl", "wb"))  
pickle.dump(classes, open("classes.pkl", "wb"))
```

- Saves the tokenizer and class list so you can use them later for predictions.

7. Build LSTM Model

```
model = Sequential()  
model.add(Embedding(input_dim=len(word_index) + 1, output_dim=64,
```

```
input_length=max_len))
model.add(LSTM(128, return_sequences=True))
model.add(Dropout(0.5))
model.add(LSTM(64))
model.add(Dropout(0.5))
model.add(Dense(len(classes), activation="softmax"))
```

- **Embedding** → Converts word numbers into dense vectors (word meaning).
- **LSTM** → Learns patterns in sequences of words.
- **Dropout** → Prevents overfitting by randomly ignoring some neurons during training.
- **Dense with softmax** → Outputs probabilities for each class (tag).

```
model.compile(loss="categorical_crossentropy", optimizer=Adam(0.001),
metrics=["accuracy"])
model.summary()
```

- Uses **categorical_crossentropy** for multi-class classification.
- Adam optimizer helps the model learn efficiently.
- `model.summary()` shows the architecture.

8. Train the Model

```
model.fit(X, y, epochs=200, batch_size=8, verbose=1)
```

- Trains the model on your dataset.
- **epochs=200** → Runs through the dataset 200 times.
- **batch_size=8** → Processes 8 samples at a time.

9. Save the Model

```
model.save("shop_bot.keras", save_format="keras")
print("✓ Model saved as shop_bot.keras")
```

- Saves the trained model in **Keras 3 format** to use later for predictions.

10. Evaluate the Model

```
y_pred_prob = model.predict(X)
y_pred = np.argmax(y_pred_prob, axis=1)
y_true = np.argmax(y, axis=1)

accuracy = accuracy_score(y_true, y_pred)
print(f"\nTraining Accuracy: {accuracy:.4f}")

print("\nClassification Report:")
print(classification_report(y_true, y_pred, target_names=classes))
```

- Predicts the outputs for the training data.
- Converts probabilities to class labels (argmax).
- Measures **accuracy** and prints **classification report** for each tag.

Summary in Simple Words

1. Load JSON dataset containing **patterns & tags**.
2. Prepare data: lowercase sentences, extract labels, list all tags.
3. Convert words into **numbers** (tokenization) and **pad sequences**.
4. Convert labels into **one-hot vectors** for multi-class classification.
5. Build an **LSTM neural network** to understand the text.
6. Train the model on your dataset.
7. Save the **model, tokenizer, and classes** for later use.

8. Evaluate model performance on the same data.

If you want, I can also **draw a simple diagram showing how this LSTM chatbot model works**, which makes it super easy to understand visually.

Do you want me to do that?